

Beyond the Casino

A Comparative Analysis of Hidden Markov Models and Gaussian Mixture Models in Regime Detection

Date: 2026/03/28

Course: ISYE 7406

Group: 99

Name: Jonathan Feng

ID Reference: 371

0 Abstract

This study investigates whether market regime detection can provide actionable signals for retail traders that seek to outperform the buy and hold benchmark or enthusiastic algorithm for traders. Using OHLCV data from SPY and QQQ spanning 2019-2026 from Alpaca API, features were engineered and derived from established quantitative finance metrics and evaluated through medallion architecture. Two unsupervised learning methods were used and compared: Hidden Markov Models and Gaussian Mixture Models. GMM was eliminated early due to instability in regime assignment, while HMM produced economically interpretable and historically accurate regimes. The average run length is 54 days and 26 transitions over the entire period. The winning configuration beat out the rest with a BIC of 29,936 with a seed of 420 using full covariance. The results suggest that HMM is effective as a regime signal detector demonstrating meaningful downside protection during high volatility periods. However, it does not outperform the buy-and-hold strategy. Infrastructure includes PostgreSQL, full medallion architecture, scheduled pipeline, and MLflow experiment tracking across 90 runs.

1 Introduction

Automated trading is a fascinating field, especially for those who know no better, or the lazy. The concept of leaving your money unattended and having it perform worse than the S&P 500 is truly what puts many minds at ease. This project takes that worry seriously. Rather than building yet another price prediction model, this project looks at a simpler question: Can knowing what state the market is currently actually help your average retail trader do better than just holding the S&P 500?

2 Problem Statement

When we talk about the state of the market, we can usually think of it in terms of categories or phases. Markets go through different types of phases, like bull runs, bear markets, and high volatility periods (Ang & Timmermann, 2012). What we name these phases are generalizations of certain behaviors and can technically be split into more specific subsets of behaviors depending on how specific you want to be.

The idea behind these phases is that we can come up with a strategy that reacts to these macro scale activities, enough so that by following it we can outperform a typical baseline of economics, the “buy and hold”. To test this, exploratory data analysis and feature

engineering will be applied to raw price data to extract meaningful signals. These features will then serve as inputs to two unsupervised learning methods: Hidden Markov Models (Rabiner, 1989) and Gaussian Mixture Models (Bishop, 2006).

The actual hypothesis is as follows: We assume that Hidden Markov Models should produce more statistically coherent regimes due to its probabilistic sequential nature, unlike its Gaussian Mixture Model counterpart which is purely distribution based. We will compare which model is best suited for this purpose and further refine the winning model.

3 Data Source

Alpaca Markets API provides historical and real-time market data and has a generous free tier (Alpaca Markets, 2026). Our data origin will come from sourcing raw market data from their historical endpoint for: open, high, low, close, volume (OHCLV) and additional elements (trade count, volume weighted average price). This will be collected daily across 2 tickers (SPY & QQQ) and cover the date range of 2019 – 2026 present, tallying to a total cumulative 3,633 rows across 8 features.

SPY and QQQ are two of the most popular exchange-traded funds (ETF) and are usually used to track the overall US stock market. SPY tracks the top 500 US companies of the broader market, and QQQ being the top 100 technology and growth stocks. Doing so will add some “spice”, since technology is known as a hyped sector that has a visible impact on the general trend of the market.

The years 2019-2026 represent an interesting time frame for our study. The specific date range covers: the 2019 late-cycle growth, 2020 COVID-19 crash and recovery, the 2021 speculative and high growth tech era, 2022 inflation and interest rate shock, the 2023-2024 bull market, and finally 2025-2026 with tariff policy uncertainties plus volatility (Federal Reserve Bank of St. Louis, 2026). We could have gone back further for more visibility, but we should also be cautious about whether this would dilute our regime detection due to irrelevant past dynamics.

4 Methodology

Infrastructure

For the general structure of this study, we will follow industry standard data engineering techniques for re-usability and transformations of data, then proceed with standard methods typical of data science: exploratory data analysis, data cleaning, feature selection, standardization, modeling, and parameter tuning.

The first step in this process will be setting the foundation, our infrastructure. The technologies I have chosen form a simple tech stack: PostgreSQL (database), Python Scripts (for transitioning between each layer of the medallion architecture), SQL (for transformations and schema defining), PowerShell Script (for our orchestration and daily execution in windows task scheduler), and Jupyter Notebook + ML-Flow (for our modeling and tracking).

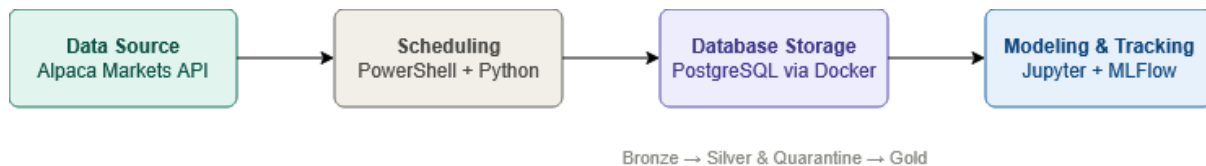


Figure 1. Infrastructure Showcase

A medallion architecture (bronze, silver, gold) is then defined which provides a separation layer for each stage where data gathers, changes, or manipulates. This allows for flexibility in our feature engineering process and provides a safe way to collect and perform data quality checks continuously.

Our bronze contains the raw landing zone for data we request from Alpaca API. This is categorized as append only and will grow day to day. Our silver layer performs data quality checks, ensuring our requests from bronze are consistent in data types and expected formats. This frequently underestimated process saves a lot of headaches down the road if data quality is imperfect, leading to lost time and frustration during the modeling phase. Imperfect or bad data will be sent to a quarantine table for further analysis.

The gold layer is where we perform feature engineering to massage the data into a useful and ingestible form for either reporting or model ingestion. When we talk about feature exploration, the OHLCV data is surface level, and non-stationary. To elaborate further, think of a \$6 dollar move in SPY when it was trading at \$200, being fundamentally different from a \$6 dollar move at \$600. We would want to normalize using a method like log returns, making the price more comparable across different time periods and price levels.

regime	regime	regime
AZ symbol	AZ symbol	AZ symbol
timestamp	timestamp	timestamp
123 open	123 open	123 open
123 high	123 high	123 high
123 low	123 low	123 low
123 close	123 close	123 close
123 volume	123 volume	123 volume
123 trade_count	123 trade_count	123 trade_count
123 vwap	123 vwap	123 vwap
processed_at	ingested_at	123 log_return
	AZ source	123 hl_range
		123 volume_ratio
		123 rolling_vol
		123 rolling_sharpe
		123 log_return_spy
		123 hl_range_spy
		123 volume_ratio_spy
		123 rolling_vol_spy
		123 rolling_sharpe_spy
		123 log_return_qqq
		123 hl_range_qqq
		123 volume_ratio_qqq
		123 rolling_vol_qqq
		123 rolling_sharpe_qqq
		123 spy_qqq_spread
		123 vol_spread
		123 spy_sharpe_ratio
		123 spy_qqq_corr
		123 vol_ratio
		123 spread_trend
		123 vol_of_vol
		123 price_ma_ratio
		123 close_open_gap
		featured_at

Figure 2. Medallion Schema (DBeaver UI)

Feature Engineering

Before we start engineering features, QQQ was merged directly to SPY rows based on daily timestamps for calculations that depend on both ticker symbols. This effectively gives us more variation to select from for the training process. The following are features that we have implemented based on metrics consistently used in quantitative analysis within the finance industry. To note, within my own code I have fixed the rolling window for all calculations to 20 to be consistent. The rolling window can be changed within the “.env” file for future experiments. *Disclaimer: I am by no means a quantitative expert, just a humble retail trader, and have explained these to the best of my knowledge.*

Log Return

How much has the price changed. The log return expresses the change in a math-friendly manner, making it easier to interpret returns across a time frame (Campbell et al., 1998). This also accounts for the non-stationary aspect that we mentioned earlier.

$$r_t = \ln(\text{Close}_t / \text{Close}_{(t-1)})$$

HL Range

Explains how wild the price was within the day. This is expressed as a percentage of the closing price. A larger range means the price was uncertain or of higher activity and vice versa (Investopedia, 2026).

$$HL_t = (High_t - Low_t) / Close_t$$

Volume Ratio

This metric explains how busy it was within the day compared to other days. This is a direct comparison of today's volume to a recent average. You can interpret this number as a higher ratio, meaning more people are trading, and lower meaning the day is calmer (Investopedia, 2026).

$$VR_t = Volume_t / RollingMean(Volume, n)$$

Rolling Volatility

This is the rolling standard deviation of the log returns over the past n periods. In short, it gauges how much returns are actually fluctuating (Campbell et al., 1998). This is a feature that is used by several other calculations within these features.

$$RVol_t = STDEV(r_t, r_{t-1}, \dots, r_{t-n+1})$$

Rolling Sharpe

In short, this metric represents how much reward you are getting per unit of risk. This is calculated over a rolling window to determine the efficiency of a stock over time (Sharpe W. F., 1994).

$$RSharpe_t = AVG(r_t, r_{t-1}, \dots, r_{t-n+1}) / STDEV(r_t, r_{t-1}, \dots, r_{t-n+1})$$

SPY – QQQ Spread

Being one of our first multi stock calculations, the SPY and QQQ spread tells us if the broader market is in line with the tech sector. An interpretation can be, when they move in sync; the spread value is close to 0. If they move stronger towards either direction, it can signal a shift in the overall investor's sentiment (Investopedia, 2026).

$$Spread_t = r_t(SPY) - r_t(QQQ)$$

Rolling Volatility Spread

This metric represents the difference between SPY and QQQ rolling volatility over the past n periods and represents which of the indexes are going through more price instability (Campbell et al., 1998). You can tell whether QQQ is absorbing more stress depending on how negative this number is.

$$\text{VolSpread}_t = \text{RVol}_t(\text{SPY}) - \text{RVol}_t(\text{QQQ})$$

SPY Sharpe Ratio

How well the overall market rewards its investors. This is a sharpe ratio that represents an overall benchmark (Sharpe W. F., 1994). When this ratio is low you can expect most stocks to move together.

$$\text{SPYSharpe}_t = r_t(\text{SPY}) / \text{RVol}_t(\text{SPY})$$

SPY – QQQ Correlation

A measure that represents whether SPY or QQQ is moving together or going in different directions. With how this equation is set up, when it ranges close to 1, they are almost moving together (Investopedia, 2026).

$$\text{rho}_t = \text{RollingCorr}(r(\text{SPY}), r(\text{QQQ}), n)$$

Spread Trend

Indicator for whether the gap between SPY and QQQ is growing or shrinking. This is a smooth spread over time. If this metric rises it means that SPY and QQQ are diverging (Investopedia, 2026).

$$\text{ST}_t = \text{RollingMean}(\text{Spread}_{(t-n+1)}, \dots, \text{Spread}_t)$$

Volatility over Volatility

Represents the rolling standard deviation of rolling volatility over the past n periods. This can tell us how much volatility is changing (Investopedia, 2026).

$$\text{VoV}_t = \text{STDEV}(\text{RVol}_{(t-n+1)}, \dots, \text{RVol}_t)$$

Price MA Ratio

This ratio calculates today's closing price divided by the rolling mean over the past n periods minus 1. It measures mean reversion, betting whether a stock is too far past its mean and will return towards its mean (Investopedia, 2026).

$$\text{PMA}_t = (\text{Close}_t / \text{RollingMean}(\text{Close}, n)) - 1$$

Close Open Gap

Measures how much a stock has moved during the trading session. A very simple metric that shows true gains or selling off, the most important aspect being it captures the intraday actions and is independent of overnight activity (Investopedia, 2026).

$$\text{Gap}_t = (\text{Close}_t - \text{Open}_t) / \text{Open}_t$$

With the features defined, the next steps in methodology will be explained as we progress through the analysis and results section. Exploratory data analysis and feature selection will be performed directly on these features, and the best subset will be selected based on variance inflation factor (VIF) and correlation thresholds (James et al., 2023).

5 Analysis and Results

To preface, this section will walk through the steps implemented in the jupyter notebook along with MLFlow model tracking (MLflow, 2026). We will go in order of standard data science practices: exploratory data analysis of the gold layer and its engineered features, data cleansing, feature selection, data splitting, scaling, and finally close out with model building (HMM, GMM) extending to parameter tuning.

Exploratory Data Analysis & Data Cleaning

Before we start cleaning through the data, a couple of insights will be implemented to ascertain and set up which variables are suitable for model ingestion further down the process. We start by reading our engineered features through polars, which is a similar entity to pandas. This library makes it easy to form connections on the fly and inherently faster during data processing, especially with its lazy evaluation method. We won't be using it extensively here since the goal is to narrow down just enough for it to be converted to pandas for "playing nice" with existing data science libraries.

Proceeding the initial data cleaning, we will be dropping all null entities due to the rolling window calculations in the gold phase. These entities will not have data due to that for the first 20 days, so they are effectively unusable. This drops our total entries down to 1,738

days, which is still plenty for our regime modeling task. Continuing, the symbol, timestamp, OHLCV, trade count, and volume weighted average price will be dropped from our set. As mentioned earlier, they are non-stationary features that only encode price levels rather than actual behavior of the market. They are not suitable for methods like GMM and HMM that are based on clustering.

Before we start culling our feature set for modeling, we should observe what the current set of data looks like distribution wise and analyze outliers and irregularities. Below is a chart created in Plotly, that charts the distribution. A separate outlier check was performed to ensure that they fall within an acceptable range per variable. The variables displayed no more than 2% of the entire data set are considered outliers on a basis of 3 standard deviations. These need to be kept since we do have world events that can explain these numbers, such as: (Covid, Rate Hikes, etc.). If we cull them without a second thought, we would be losing very important historical information.

Feature	Count
log_return	26
hl_range	30
volume_ratio	31
rolling_vol	45
rolling_sharpe	4
spy_qqq_spread	21
vol_spread	8
spy_sharpe_ratio	11
spy_qqq_corr	35

Table 1. Outlier Counts

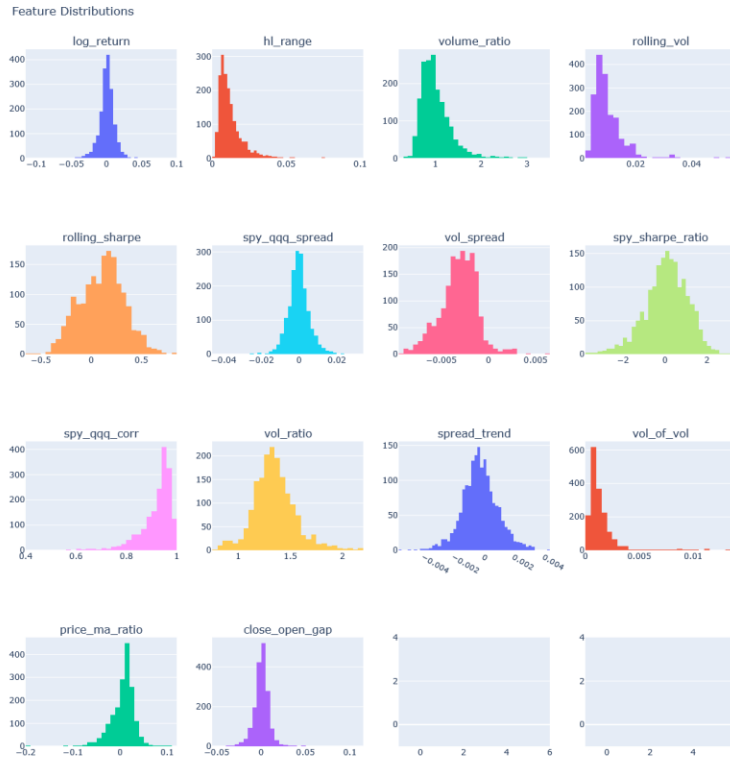


Fig 3. Feature Distributions

Feature Selection

Given that overall features are not taken out from the processes above, we can finally look to lessen the number of variables through methods like covariance threshold and variance inflation factor. The correlation threshold is simpler of the two. We will be looking at the pairwise relationship between each feature against all others (James et al., 2023). If two of them are above our threshold 0.85, it is basically the same as saying those two variables are redundant. The feature correlation matrix plotted in (Fig 4.) shows just that. The goal here is to not over-complicate the model.

VIF serves a similar purpose but goes even further and catches multi-collinearity. One feature may be okay against all others in isolation, but we can't entirely discount that a combination of features could also reconstruct the same variable (James et al., 2023). After performing these steps, we obtain a total of 14 features. We have effectively narrowed down more than half the features we started out with, and the remaining are mostly feature engineered ones, giving more validity to using industry standard metrics.

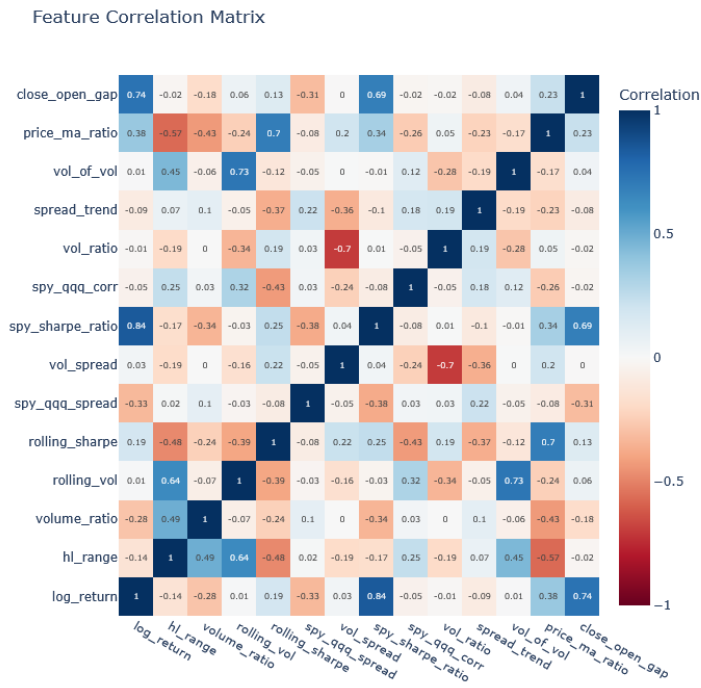


Fig 4. Feature Correlation Matrix

Feature	VIF
rolling_vol	5.349474
log_return	4.732691
vol_spread	4.164371
spy_sharpe_ratio	4.148099
vol_ratio	4.084204
hl_range	3.949969
rolling_sharpe	3.30773
price_ma_ratio	3.191088
vol_of_vol	2.543715
close_open_gap	2.378934
volume_ratio	2.154231
spread_trend	1.473445
spy_qqq_corr	1.312418
spy_qqq_spread	1.23441

Table 2. VIF Culling

To further validate the data itself before we move on to the modeling phase, we will graph some important features over time to show whether it captures some expected and obvious regimes already. Below, we plot the rolling volatility, log returns, and SPY on QQQ correlations. The two most obvious things to look for here are visible effects of events like the COVID spike and perhaps even the most recent tariff uncertainty of 2025-2026. While events between 2022- 2025 look relatively calm compared to other years we cannot come to any immediate conclusions that there aren't regimes throughout. The two most obvious historical events do show up especially COVID; you can see it on the rolling volatility and log return sections of the time series chart.

To justify the usage of QQQ tech sector metrics, I decided to include the correlation to show that there are multiple dips within this metric that suggest tech and overall market has been running independently multiple times over this 7-year span, justifying our cross ticker features for our model input.

In addition to a time series plot, an extra step was inserted to physically show auto correlation levels; this is displayed across the same three variables on auto correlation function (ACF) and displayed across 40 lags. Rolling volatility and correlation of SPY AND QQQ exhibits slow decays, while log returns drop off immediately. The slowness of the decay shows that today's value is strongly influenced by recent history.

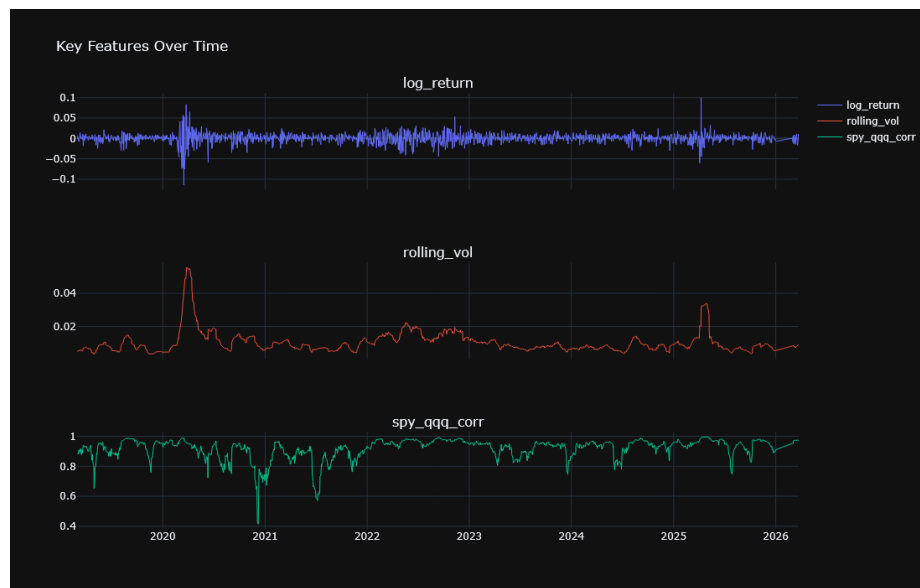


Fig 5. Time series trend 3 features

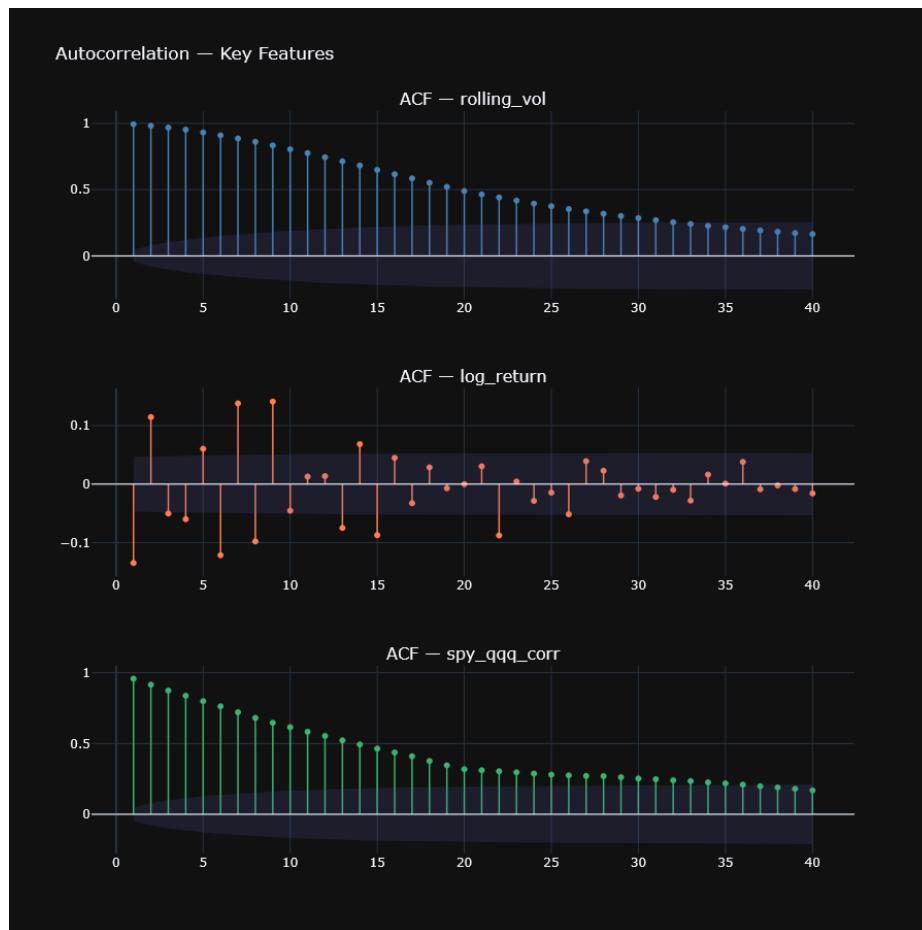


Fig 6. Auto Correlation 3 Features

Modeling

Before the modeling begins, we will put the data through a split for training and testing data. An 80-20 split is used, preserving the ordering since this is a time series. An additional scaled form of this data is fit to the train subset only to prevent data leakage from test set onto preprocessing. Even though most of our data exploration results are suggesting that HMM would be a better fit let me explain to you what our two unsupervised models are. To note, we will also be using fixed parameters for the first pass through results, and I will have plans to try out more parameter tuning for the final phase.

Hidden Markov Model (HMM) assumes that a market has multiple hidden states, and it operates directly through it. Observations are state-dependent and are assumed to be attributed to one. HMM assumes a high level of sequential awareness, and each state is modeled as a multivariate Gaussian (Rabiner, 1989).

$$P(O | \lambda) = \sum_Q \pi_{\{q_1\}} \cdot \prod a_{\{q_{t-1}, q_t\}} \cdot \prod b_{\{q_t\}}(o_t)$$

Gaussian Mixture Model (GMM) is purely distributional and models the overall set as a Gaussian distribution and assigns observations to whichever it most likely identifies as. GMM has no concept of sequence or time; each is evaluated independently; this will serve as our baseline (Bishop, 2006).

$$p(x) = \sum \pi_k \cdot N(x; \mu_k, \Sigma_k)$$

To evaluate which model produces better regimes, several metrics are calculated. Log-likelihood, Bayesian Information Criterion (BIC) (Schwarz, 1978), transition count, average regime run length, and annualized Sharpe ratio per regime, and max + average drawdown to compare to the buy and hold benchmark. Log-likelihood is used to determine how well the model learned the parameters, measured by the actual fit of the data, and is good for comparing against the same model types. BIC extends log-likelihood but penalizes the model for complexity. We will be using BIC as our main model evaluator to assess model fit. Average run days and transitions provide useful information on how much the model switches regimes and how frequently. A very common metric applied in this field is drawdown; they calculate the worst peak to trough loss throughout a period, maximum being the absolute worst and average being a more accurate measure of all drawdown periods. Crisis Sharpe measures risk-adjusted return during the bear regime specifically, with equivalent Sharpe calculations applied to all identified regimes.

An initial pass of HMM and GMM implementations was based on `covariance_type = full`, `n_iter=100`, `n_init=10`, and `n_components=3`. GMM was eliminated early as it consistently underperformed across initial configurations, and subsequent analysis focuses solely on HMM tuning.

To fine tune HMM I have decided to implement a custom cross validation, since MLFlow does not have a direct built-in functionality for HMM specifically. We will be testing on `n_components` (our regime buckets), three different covariance types, three different `n_its`, and five random states.

Parameter	Values Searched
<code>n_components</code>	3, 4
<code>covariance_type</code>	full, tied, diag
<code>n_iter</code>	100, 200, 300
<code>random_state</code>	420, 123, 1337, 69, 808

Table 3. Grid Search CV Parameters

The reasoning behind the `n_components` was to test whether the model would perform better given more choice. Two components were deemed insufficient to capture meaningful regime variability, and anything beyond four would be too sensitive. Covariance type determines how the model estimates the shape of each regime's Gaussian distribution. Full covariance determines uniqueness across each regime and allows for more complex relationships to be captured, coming at the cost of more parameters. Tied constrains all regimes to share the same matrix, while diagonal assumes the features are independent within each regime. Iterations control the maximum number of Baum-Welch EM iterations allowed during fitting. The higher the values, the more opportunity there is for the model to converge. 100, 200, and 300 were selected to assess whether additional iterations improved convergence. Lastly, the random state accounts for initialization sensitivity. Since HMM fitting via EM is not guaranteed to find the global optimum, we need to account for the possibility of convergence to different local optima. Testing these seeds provides us with confidence that a selected configuration is stable, and not because of favorable initialization.

Results

The top six results are shown out of the 90 runs performed via cross validation. These results are directly drawn from the experiments of MLFlow statistics tracking and tabled based on our evaluation metrics, ranked by BIC (our primary selection criterion).

	Run Name	Log-Likelihood	BIC	Avg Run Days	Transitions	MDD	Avg DD	Crisis Sharpe
1	HMM_n3_full_iter100_seed420	-1.917e+07	29,936	54.0	26	27.1%	7.8%	0.482
2	HMM_n3_full_iter200_seed420	-1.917e+07	29,936	54.0	26	27.1%	7.8%	0.482
3	HMM_n3_full_iter300_seed420	-1.917e+07	29,936	54.0	26	27.1%	7.8%	0.482
4	HMM_n3_full_iter100_seed123	-1.955e+07	30,470	54.0	26	27.1%	7.5%	0.06
5	HMM_n3_full_iter100_seed69	-1.954e+07	30,454	50.2	28	27.1%	7.6%	0.16
6	HMM_n3_full_iter200_seed69	-1.954e+07	30,454	50.2	28	27.1%	7.6%	0.16

Table 4. Grid search CV HMM Results (Ranked By: Lowest BIC)

Regime	Label	Mean Return	Sharpe
0	Bull	0.000874	2.073
1	Transitional	-0.000033	-0.039
2	Crisis	0.000937	0.482

Table 5. Winning HMM Label, Mean Return, and Sharpe

Regime	Label	Std Dev	Count
0	Bull	0.006985	256
1	Transitional	0.013323	69
2	Crisis	0.027585	27

Table 6. Winning HMM Distribution of Label Counts

The tabulated results show that runs 1-3 are identical and give us key information that the `n_iter` field does not matter past convergence; anything past 100 didn't change anything. Seed 420 demonstrated stability by reproducing identical results across three runs. Our winning combination of parameters is `seed=420`, `n=3` components, full covariance, and `n_iter=100`. This produced the lowest BIC at ~29,936 meaning it has the best fit with the least complexity penalty applied. Full covariance gave it enough flexibility to accurately capture regime shapes without overfitting.

The average run days landing around ~54 show us that these are not noisy regimes, and the pattern sticks. This also extends to the number of transitions landing at 26. These are the total amount of times the regime switched throughout the span of this seven-year period. This is extremely important because if the model switches constantly, it would be useless in practice and highly unpredictable. You can clearly see why that is in the GMM results plot below. The average run days are very short and end up forming a bar code pattern, exhibiting instability in capturing regime shapes. In practice, using results such as these would accrue substantial transaction costs when used as part of an automated system. Maximum draw down at the single worst peak was equal across the tabulated results, -27.1% is the worst it ever got. Our actual benchmark draw down recorded was -34.2% at worst. While the average draw down -7.8% does not outperform our benchmark's -7.3%, our maximum would provide us with a meaningful signal for downside protection in one of the most stressful periods in this period "covid market crash".

Individual regime Sharpes were calculated for all regimes and can be seen clearer in the smaller tables. The bull regime had a Sharpe of 2.073 and signals that there is strong positive risk adjusted returns meaning a healthy market state. The transitional regime is -0.039 being essentially flat. This is neither good nor bad and considered an in between state. The crisis regime had a Sharpe of 0.482 which is surprising considering most would expect a negative Sharpe in a crisis regime; the positive value instead suggests volatility clustering rather than sustained negative returns. High volatility periods can still have positive returns; they would just have wilder swings and are more apparent when regime labels are plotted against the price trend.

It is important to note that regime labels are not consistent across runs. The integer labels are assigned to bull, crisis; transitional can vary on the initialization and that is the expected behavior in unsupervised models. HMM has no default ordering of states, and labels are assigned based on mean returns. One additional thing worth noting is the seed 420 consistently converging to the lowest BIC across all iteration counts. This demonstrates that this specific initialization found a more optimal solution than other seeds. This matters since HMM fitting is not guaranteed to find the global optimum and depends on where it starts.

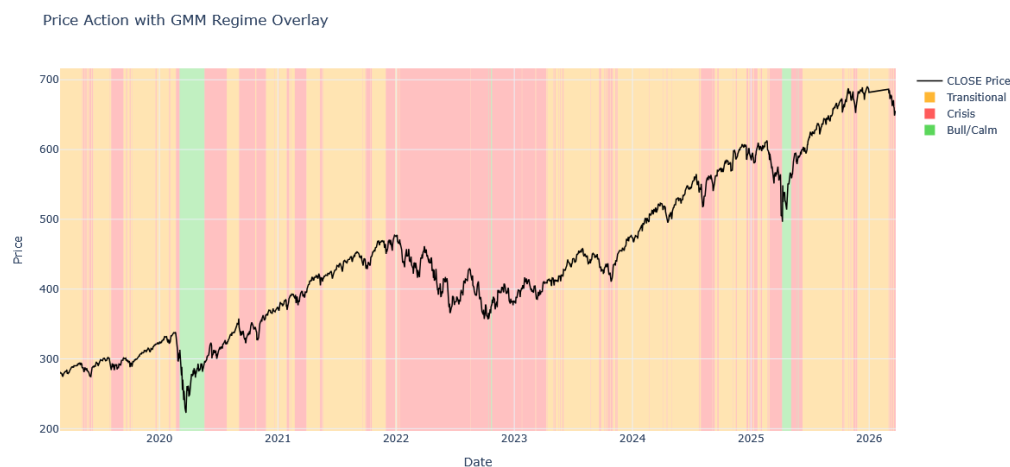


Fig 7. GMM Results with Regime Labeling

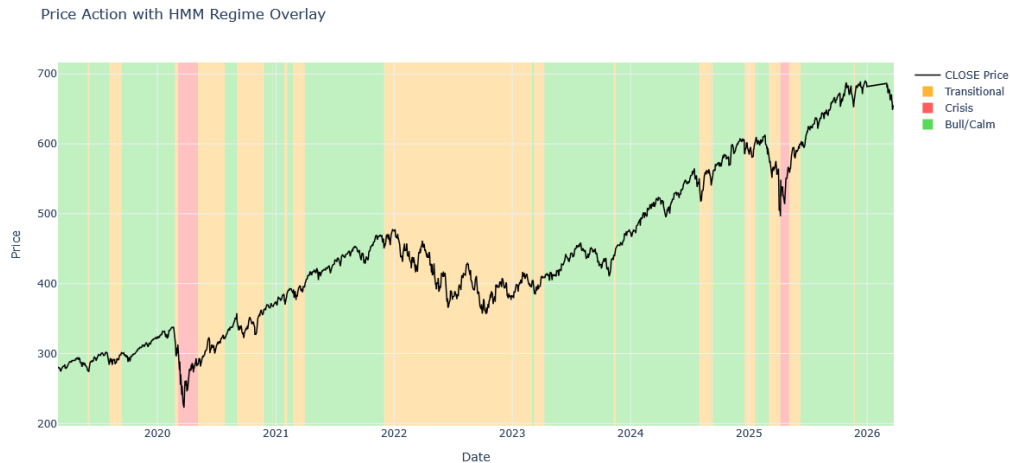


Fig 8. HMM Results with Regime Labeling

The plots above show the identified regimes overlaid on the SPY price trend across our 2019-2026 timeline. You can immediately notice what I was referring to by “barcoding” in the GMM plot, oversensitivity to regime trends and inaccurate identifications of crisis, transitional, and bull regimes. The HMM plot shows a night and day difference, with minimal regime switching and longer average running days indicating that the model learned something. We can also glean the two sharpest drops labeled as our crisis, having accurately identified it as a high volatility period, perfectly matching up with periods of historical events most notably the COVID crash of early 2020 and the 2022 rate hike driven selloff.

6 Conclusions

Coming back around, our hypothesis states that HMM would produce more statistically and coherent regimes than GMM due to its sequential nature, and the results confirmed that. GMM was eliminated early across all configurations due to HMM producing more stable and economically interpretable regimes with meaningful run lengths. The barcoding pattern in GMM versus the clean regime separation in HMM is our visual proof.

The model did not conclusively outperform the buy and hold as standalone. The average drawdown was comparable to our benchmark, and the crisis Sharpe was unexpectedly positive. However, the maximum drawdown gap suggests that there is meaningful regime awareness that can prove to be useful during high volatility for downside protection. This leads me to believe that HMM is useful as a signal layer but not a standalone strategy. It is useful at detecting regime shifts and volatility and is good at telling us when markets are behaving unusually. This could be used in conjunction with other market signals to

form a more coherent trading strategy or even be used as a reliable labeling method in training other models like XGBoost.

Learned Outcomes

I learned multiple obvious and non-obvious lessons throughout this project both on the infrastructure and modeling side. GMM was the right baseline call and learned that sequential awareness matters in time series data, confirmed by my HMM results. Initialization sensitivity is real and is worth testing multiple seeds because convergence stability isn't guaranteed. It was a waste computing n iterations past a certain point, possibly worth an initial test.

MLflow experiment tracking was incredibly useful and provided a standardized way to run and log 90 runs. This result was query-able at the end, making model selection, comparison, and charting a clean process. Even though the infrastructure was a pain to setup for the scope of this project, it was worth it because I can see the same framework being re-used in other projects that utilize Alpaca API.

Although SPY is a good representation, it limits generalizability. Having multi-assets would have strengthened the findings. The crisis regime labeling by mean return alone is imperfect. Volatility would have been more appropriate. This entire project is better framed as building a regime signal rather than a trading strategy, which wasn't fully clear at the start.

Additional course comments:

The course structure feels heavily loaded in terms of homework delivery, especially considering the group project timeframe given. The content was good, but the structure became redundant after a while. I get the point is to train a habit of writing well while applying concepts, but this could have been done more tastefully. Also, the time framing would make more sense as content, videos, quizzes, homework for all 5 first and use the final take-home as a midterm, then the project should be the deliverable as a final.

Order of Credits

- APPENDIX A – Docker Compose & MLFlow UI
- APPENDIX B – Gold Layer (Feature Engineering)
- APPENDIX C – Data Cleaning (Exploration & Visualization Not Included)
- APPENDIX D – Modeling & MLFlow
- BIBLIOGRAPHY

6 Appendix

APPENDIX A – Docker Compose & MLFlow UI

services:

postgres:

image: postgres:16

container_name: trading_db

env_file: .env

environment:

PGDATA: /var/lib/postgresql/data/pgdata

ports:

- "5432:5432"

volumes:

- pgdata:/var/lib/postgresql/data

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \$POSTGRES_USER"]

interval: 10s

timeout: 5s

retries: 5

volumes:

pgdata:

*mlflow ui --backend-store-uri sqlite:///data/mlflow/mlflow.db > data/mlflow/mlflow.log
2>&1 &*

APPENDIX B – Gold Layer (Feature Engineering)

```
from alpaca.data.historical import StockHistoricalDataClient from alpaca.data.requests import
StockBarsRequest from alpaca.data.timeframe import TimeFrame
```

```
from datetime import datetime, timedelta, timezone
from dotenv import load_dotenv
import polars as pl
import psycopg2
import os
import logging
```

load global variables

```
load_dotenv()
DB_URL = f"postgresql://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}@localhost:5432/{os.getenv('POSTGRES_DB')}"
GOLD_TABLE=os.getenv("GOLD_TABLE")
SILVER_TABLE=os.getenv("SILVER_TABLE")
ROLLING_WINDOW=int(os.getenv("ROLLING_WINDOW"))
#Load Logger
os.makedirs("./pipeline/logs", exist_ok=True)
```

setting up logging config

```
logging.basicConfig( level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
handlers=[ logging.FileHandler("./pipeline/logs/silver_to_gold.log"), logging.StreamHandler() ] )
```

```
log = logging.getLogger(name)
```

create silver schema if not exists

```
def startup():
    with open("./data/sql/gold_schema.sql", "r") as f:
        sql = f.read()
    with psycopg2.connect(DB_URL) as conn:
        with conn.cursor() as cur:
            cur.execute(sql)
            cur.execute(f"SELECT COUNT(*) FROM {GOLD_TABLE};")
            count = cur.fetchone()[0]
        conn.commit()

    return count > 0
```

get new silver entries that are not in gold

```
def get_silver_entries():
    res = pl.read_database_uri(
        query=f"""
        SELECT *
        FROM silver.regime
        WHERE timestamp >= (
            SELECT COALESCE(MAX(timestamp), '1900-01-01'::timestamp) - INTERVAL
            '{ROLLING_WINDOW} days'
        """
```

```

        FROM gold.regime
    )
    """
    uri=DB_URL
)

return res

calculate all new fields

def feature_engineering(df):

df = df.sort(["symbol", "timestamp"])

# log return
# additive returns over time that are normally distributed.
df = df.with_columns(
    (pl.col("close") /
pl.col("close").shift(1).over("symbol")).log().alias("log_return")
)

# hl range (high low spread)
# this is also known as intraday range, the thought is the wider the range the more
the uncertainty.
df = df.with_columns(
    ((pl.col("high") - pl.col("low")) / pl.col("close")).alias("hl_range")
)

# volume ratio
# volume over rolling average. raw volume by itself not very useful. Tells you
whether or not
# the participation in the stock is abnormally high or low relative to its own norm.
df = df.with_columns(
    (pl.col("volume") /
pl.col("volume").rolling_mean(window_size=ROLLING_WINDOW).over("symbol")).alias("volu
me_ratio"),
)

# rolling volume
# rolling standard deviation of log returns. A fundamental indicator for quants, low
volume
# behaves differently than high volume and this is an indicator to capture that
behavior
df = df.with_columns(

pl.col("log_return").rolling_std(window_size=ROLLING_WINDOW).over("symbol").alias("ro

```

```

    lling_vol")
)

# rolling sharpe
# mean return dvided by rolling standard deviation. Used to distinguish between low
volume (good sharpe)
# or zero to negative sharpe that indicates quality of return.
df = df.with_columns(
    (pl.col("log_return").rolling_mean(window_size=ROLLING_WINDOW).over("symbol") /
     pl.col("log_return").rolling_std(window_size=ROLLING_WINDOW).over("symbol")
    ).alias("rolling_sharpe"),
)

# Pull out individual features and join them on a wide form
parsed_spy = df.filter(pl.col("symbol") == "SPY").select([
    "timestamp",
    pl.col("log_return").alias("log_return_spy"),
    pl.col("hl_range").alias("hl_range_spy"),
    pl.col("volume_ratio").alias("volume_ratio_spy"),
    pl.col("rolling_vol").alias("rolling_vol_spy"),
    pl.col("rolling_sharpe").alias("rolling_sharpe_spy"),
])

parsed_qqq = df.filter(pl.col("symbol") == "QQQ").select([
    "timestamp",
    pl.col("log_return").alias("log_return_qqq"),
    pl.col("hl_range").alias("hl_range_qqq"),
    pl.col("volume_ratio").alias("volume_ratio_qqq"),
    pl.col("rolling_vol").alias("rolling_vol_qqq"),
    pl.col("rolling_sharpe").alias("rolling_sharpe_qqq"),
])

df = df.join(parsed_spy, on="timestamp", how="left")
df = df.join(parsed_qqq, on="timestamp", how="left")

# calculated metrics between both

# spy - qqq spread
# this can tell us the risk signal. When tech leads QQQ this is negative spread and
it means
# people want to be more risky for growth, and the opposite is true for conservatism
df = df.with_columns(
    (pl.col("log_return_spy") - pl.col("log_return_qqq")).alias("spy_qqq_spread")
)

```

```

# rolling volume spread
# Difference in volatility of spy and qqq. this calculates how disproportionate each
are from each other
# used as early indicator because you will see a larger negative given by QQQ *tech
movements effect the market
df = df.with_columns(
    (pl.col("rolling_vol_spy") - pl.col("rolling_vol_qqq")).alias("vol_spread")
)

# spy sharpe ratio
# a simple grounder for whether the market is in a rewarding or chaotic phase. When
the sharpe is bad it means individual
# stocks all move together.
df = df.with_columns(
    (pl.col("log_return_spy") / pl.col("rolling_vol_spy")).alias("spy_sharpe_ratio")
)

# spy + qqq correlation
# just tracks whether or not spy or qqq is moving similarly. when it diverges it
usually means a regime is changing
# because money is moving between sectors.
df = df.with_columns(
    pl.rolling_corr(pl.col("log_return_spy"), pl.col("log_return_qqq"),
window_size=ROLLING_WINDOW).alias("spy_qqq_corr"),
    (pl.col("rolling_vol_qqq") / pl.col("rolling_vol_spy")).alias("vol_ratio"),
)

# spread trend
# tells us whether one or the other is beating each other consistently. If there is a
sustained trend it may
# signal a regime change
df = df.with_columns(
    pl.col("spy_qqq_spread").rolling_mean(window_size=ROLLING_WINDOW).alias("spread_trend")
)

# vol_of_vol
# tells us how consistently volatile a market is, steady vs erratic
df = df.with_columns(
    pl.col("rolling_vol").rolling_std(window_size=ROLLING_WINDOW).over("symbol").alias("vol_of_vol")
)

# this is basically rolling mean reversion is it coming back to its mean or is it

```



```

consistently above or below average
df = df.with_columns(
    (pl.col("close") /
pl.col("close").rolling_mean(window_size=ROLLING_WINDOW).over("symbol") -
1).alias("price_ma_ratio")
)

# it just means did the stock go up or down today during the trading day. tells you
more so the overall price action.
df = df.with_columns(
    ((pl.col("close") - pl.col("open")) / pl.col("open")).alias("close_open_gap"),
)

return df.filter(pl.col("symbol") == "SPY")

main process

if name == "main":

    try:
        log.info("Starting silver to gold pipeline.")
        startup()
        new_df = get_silver_entries()
        log.info(f"Silver entries detected: {len(new_df)}")

        if len(new_df) == 0:
            log.info("No new silver entries for processing. Exiting process.")
            exit()

        log.info(f"Starting feature engineering.")

        result_df = feature_engineering(new_df)

        log.info(f"Finished writing new features.")

        result_df = result_df.drop(["processed_at"]).with_columns([
            pl.lit(datetime.now(timezone.utc)).alias("featured_at")
        ])

        log.info(f"Try writing records to gold.")

        existing_gold_keys = pl.read_database_uri(
            query=f"SELECT symbol, timestamp FROM {GOLD_TABLE}",
            uri=DB_URL
        )

```

```

to_append = result_df.join(
    existing_gold_keys,
    on=["symbol", "timestamp"],
    how="anti"
)

if to_append.is_empty():
    log.info("No new unique records to append. Exiting.")
    exit()

log.info(f"Appending {len(to_append)} new unique records to gold.")

to_append.write_database(
    table_name=GOLD_TABLE,
    connection=DB_URL,
    if_table_exists="append"
)

log.info(f"Gold Process Finished.")

except Exception as e:
    log.error(f"Gold pipeline failed: {e}", exc_info=True)
    raise

```

APPENDIX C – Data Cleaning (Exploration & Visualization Not Included)

```

from dotenv import load_dotenv
import polars as pl
import pandas as pd
import os
import mlflow
import mlflow.sklearn
from mlflow.tracking import MlflowClient
from datetime import date
import scipy.stats as stats
import matplotlib.pyplot as plt

# load global variables
load_dotenv()

```

```

DB_URL =
f"postgresql://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}@localhost:5432/{os.ge
tenv('POSTGRES_DB')}"
GOLD_TABLE=os.getenv("GOLD_TABLE")

gold_df = pl.read_database_uri(
    query=f"SELECT g.* FROM {GOLD_TABLE} as g",
    uri=DB_URL
)

display(gold_df.tail(5))

# drop all na rows
df = gold_df.to_pandas().dropna()

# drop metadata and ohlcv columns
drop_default_cols= [
    'symbol', 'featured_at', 'open', 'high', 'low', 'close', 'volume', 'trade_count', 'vwap'
]
df = df.drop(columns=drop_default_cols)

# correlation cull
import numpy as np

def correlation_cull(df, features, threshold=0.85):
    c_matrix = df[features].corr().abs()
    c_u = c_matrix.where(
        np.triu(np.ones(c_matrix.shape), k=1).astype(bool)
    )

    drop_logic = [c for c in c_u.columns if any(c_u[c] > threshold)]

    return [f for f in features if f not in drop_logic], drop_logic

corr_select, corr_discard = correlation_cull(df, df.columns, 0.85)
print(corr_discard)
print(corr_select)
print(len(corr_select))

# vif drop
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.preprocessing import StandardScaler

def vif_cull(df, features, threshold=10):

```

```

remaining = features.copy()

while True:
    X = df[remaining].dropna()
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    vif_data = pd.DataFrame({
        'feature': remaining,
        'VIF': [variance_inflation_factor(X_scaled, i)
                 for i in range(X_scaled.shape[1])]
    }).sort_values('VIF', ascending=False)

    max_vif = vif_data.iloc[0]['VIF']
    max_feat = vif_data.iloc[0]['feature']

    if max_vif > threshold:
        print(f"Dropping '{max_feat}' with VIF = {max_vif:.2f}")
        remaining.remove(max_feat)
    else:
        break

print(f"\nFinal VIF table:\n{vif_data}")
return remaining

corr_select.remove("timestamp")
feature_cols_final = vif_cull(df, corr_select, threshold=10)
print(feature_cols_final)
print(len(feature_cols_final))

feature_cols_final_no_ts = [col for col in feature_cols_final if col != 'timestamp']
split_idx = int(len(df) * 0.8)

train_df = df.iloc[:split_idx]
test_df = df.iloc[split_idx:]

print(f"Train: {train_df['timestamp'].min()} to {train_df['timestamp'].max()} — {len(train_df)} rows")
print(f"Test: {test_df['timestamp'].min()} to {test_df['timestamp'].max()} — {len(test_df)} rows")

X_train = train_df[feature_cols_final_no_ts].values
X_test = test_df[feature_cols_final_no_ts].values

# scale AFTER split, fit only on train

```

```

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

APPENDIX D – Modeling & MLFlow

call mlflow

```

from sklearn.linear_model import LogisticRegression from sklearn.tree import DecisionTreeClassifier from
sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import TimeSeriesSplit,
GridSearchCV from sklearn.metrics import accuracy_score, roc_auc_score, confusion_matrix import
xgboost as xgb import mlflow, mlflow.sklearn import os

```

initiate tracking and experiment setting for mlflow

```

#mlflow.set_tracking_uri("http://localhost:5000")

os.chdir("./ISYE-7406-OAN/Project") print(os.getcwd()) mlflow.set_tracking_uri("sqlite:///.../Gtech/ISYE-
7406-OAN/Project/data/mlflow/mlflow.db") exp = mlflow.set_experiment("regime_spy_qqq")

def plot_regime_overlay(df, label_col, price_col='close', model_name="HMM", crisis_regime=1,
bull_regime=2, trans_regime=0):

# Regime Overlay plotting for final results - cut from final paper

def log_regime_model(model, model_name, X_train, X_test, train_df, test_df, base_full_df,
model_type="HMM"):

with mlflow.start_run(run_name=model_name):

    # PREDICT & AUTO-DETECT CRISIS
    regime_col = f"{model_name.lower()}_regime"
    train_df[regime_col] = model.predict(X_train)

    regime_col = f"{model_type.lower()}_regime"
    train_df = train_df.copy()
    test_df = test_df.copy()
    train_df[regime_col] = model.predict(X_train)
    test_df[regime_col] = model.predict(X_test)
    full_df = pd.concat([train_df,
test_df]).sort_values('timestamp').reset_index(drop=True)
    full_df = full_df.merge(base_full_df[['timestamp', 'close']], on='timestamp',
how='left')

```

```

full_df[regime_col] = full_df[regime_col].astype(int)

# PARAMS
mlflow.log_param("model_type", model_name)
mlflow.log_param("n_components", model.n_components)
mlflow.log_param("covariance_type", model.covariance_type)

# BIC & LOG LIKELIHOOD
if model_name == "HMM":

    n_samples, n_features = X_train.shape
    n_states = model.n_components
    cov_type = model.covariance_type

    if cov_type == 'full':
        cov_params = n_states * n_features * (n_features + 1) / 2
    elif cov_type == 'tied':
        cov_params = n_features * (n_features + 1) / 2
    elif cov_type == 'diag':
        cov_params = n_states * n_features
    elif cov_type == 'spherical':
        cov_params = n_states

    k = (n_states**2 - n_states) + (n_states - 1) + (n_states * n_features) +
cov_params
    log_lik = model.score(X_train)
    bic = -2 * log_lik + k * np.log(n_samples)

else:
    log_lik = model.score(X_train) * len(X_train)
    bic = model.bic(X_train)

mlflow.log_metric("log_likelihood", log_lik)
mlflow.log_metric("bic", bic)

# STABILITY
labels = train_df[regime_col].values
transitions = (labels[1:] != labels[:-1]).sum()
avg_run = len(labels) / transitions if transitions > 0 else len(labels)

mlflow.log_metric("n_transitions", transitions)
mlflow.log_metric("avg_run_length_days", avg_run)

# SHARPE BY REGIME
stats = train_df.groupby(regime_col)['log_return'].agg(['mean', 'std'])

```

```

sharpes = (stats['mean'] / stats['std']) * np.sqrt(252)
for regime_id, sharpe_val in sharpes.items():
    mlflow.log_metric(f"sharpe_regime_{regime_id}", round(sharpe_val, 3))

crisis_regime = stats['std'].idxmax()
bull_regime = (stats['mean'] / stats['std']).idxmax()
trans_regime = [r for r in stats.index if r not in [crisis_regime,
bull_regime]][0]

# MDD & AVG DRAWDOWN
strat_ret = np.where(train_df[regime_col] == crisis_regime, 0,
train_df['log_return'])
bench_cum = train_df['log_return'].cumsum().apply(np.exp)
strat_cum = pd.Series(strat_ret).cumsum().apply(np.exp)

bench_dd = (bench_cum / bench_cum.cummax()) - 1
strat_dd = (strat_cum / strat_cum.cummax()) - 1

bench_mdd = bench_dd.min()
strat_mdd = strat_dd.min()
bench_avg_dd = bench_dd[bench_dd < 0].mean()
strat_avg_dd = strat_dd[strat_dd < 0].mean()

mlflow.log_metric("benchmark_mdd", bench_mdd)
mlflow.log_metric("strategy_mdd", strat_mdd)
mlflow.log_metric("benchmark_avg_drawdown", bench_avg_dd)
mlflow.log_metric("strategy_avg_drawdown", strat_avg_dd)

# log all three regime labels so you can verify in mlflow ui
mlflow.log_param("crisis_regime_label", int(crisis_regime))
mlflow.log_param("bull_regime_label", int(bull_regime))
mlflow.log_param("trans_regime_label", int(trans_regime))

# addition labels statistics

regime_means = train_df.groupby(regime_col)['log_return'].mean()
for regime_id, mean_val in regime_means.items():
    mlflow.log_metric(f"mean_return_regime_{regime_id}", round(mean_val, 6))

# graphing
if model.n_components == 3:

```

```

fig_hmm = plot_regime_overlay(full_df, 'hmm_regime', model_name="HMM",
                             crisis_regime=crisis_regime, bull_regime=bull_regime, trans_regime=trans_regime)
mlflow.log_figure(fig_hmm, "regime_overlay_hmm.html")

from sklearn.mixture import GaussianMixture

from hmmlearn.hmm import GaussianHMM

import plotly.graph_objects as go

hmm = GaussianHMM(
    n_components=3,
    covariance_type='full',
    n_iter=100,
    random_state=42
)
hmm.fit(X_train_scaled)

gmm = GaussianMixture(
    n_components=3,
    covariance_type='full',
    n_init=10,
    random_state=42
)
gmm.fit(X_train_scaled)

# Full df for plotting
test_df = test_df.copy()
test_df['hmm_regime'] = hmm.predict(X_test_scaled)

```



```

test_df['gmm_regime'] = gmm.predict(X_test_scaled)

full_df = pd.concat([train_df,
test_df]).sort_values('timestamp').reset_index(drop=True)

price_df = gold_df.to_pandas()[['timestamp', 'close']].dropna()
full_df = full_df.merge(price_df, on='timestamp', how='left')

full_df = full_df.dropna(subset=['hmm_regime']).reset_index(drop=True)
full_df['hmm_regime'] = full_df['hmm_regime'].astype(int)

# log metrics
hmm_run_id = log_regime_model(hmm, "HMM", X_train_scaled, train_df, full_df)
gmm_run_id = log_regime_model(gmm, "GMM", X_train_scaled, train_df, full_df)

# grid search parameters
param_grid = {
    'n_components':    [4],
    'covariance_type': ['full', 'tied', 'diag'],
    'n_iter':          [100, 200, 300],
    'random_state':    [420, 123, 1337, 69, 808]
}

keys = list(param_grid.keys())
values = list(param_grid.values())

# base full_df with price – built once, regimes get added per run inside the function

```

```

price_df = gold_df.to_pandas()[['timestamp', 'close']].dropna()

base_full_df = pd.concat([train_df,
test_df]).sort_values('timestamp').reset_index(drop=True)

base_full_df = base_full_df.merge(price_df, on='timestamp', how='left')

for combo in product(*values):

    params = dict(zip(keys, combo))

    model = GaussianHMM(
        n_components=params['n_components'],
        covariance_type=params['covariance_type'],
        n_iter=params['n_iter'],
        random_state=params['random_state']
    )

    try:
        model.fit(X_train_scaled)
    except Exception as e:
        print(f"Failed: {params} | {e}")
        continue

    if not model.monitor_.converged:
        print(f"Did not converge: {params}")
        continue

    run_name =
f"HMM_n{params['n_components']}__{params['covariance_type']}_iter{params['n_iter']}_se
ed{params['random_state']}"

```

```
log_regime_model(model, run_name, X_train_scaled, X_test_scaled, train_df,
test_df, base_full_df.copy())
```

7 Bibliography

Alpaca Markets. (2026). Alpaca Markets API documentation. Retrieved from <https://docs.alpaca.markets>

Ang, A., & Timmermann, A. (2012). Regime changes and financial markets. *Annu. Rev. Financ. Econ.*

Bishop, C. M., & Nasrabadi, N. M. (2006). *Pattern recognition and machine learning*. New York: springer.

Campbell, J. Y., Lo, A. W., MacKinlay, A. C., & Whitelaw, R. F. (1998). The econometrics of financial markets. *Macroeconomic Dynamics*

Federal Reserve Bank of St. Louis. (2026). FRED Economic Data. Retrieved from <https://fred.stlouisfed.org>

hmmlearn developers. (2026). hmmlearn documentation. Retrieved from <https://hmmlearn.readthedocs.io>

Investopedia. (2026). Retrieved from <https://www.investopedia.com>

James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). Statistical learning. In *An introduction to statistical learning: With applications in Python* Cham: Springer International Publishing.

MLflow. (2026). MLflow documentation. Retrieved from <https://mlflow.org>

Pedregosa, F., Varoquaux, G., Cournapeau, D., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*

Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2)

Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2)

Sharpe, W. F. (1994). The sharpe ratio, the journal of portfolio management.